

Ayudantía 3

Arquitectura de Software

Modelo Repositorio, SOA

— Primer Semestre 2026
Clase 29/04

Ayudante

Bruno Rosales

bruno.rosales@mail.udp.cl

Profesor

Juan Ricardo Giadach

Temario

01

Repaso

SOA y Modelo de Repositorio

02

Proyecto SOA

Código de ejemplo

SOA (Service Oriented Architecture)

Definición

- Es una **evolución del modelo Cliente/Servidor** que se enfoca en la **creación de servicios modulares e independientes** para construir un sistema.
- La comunicación entre los servicios y los clientes se realiza a través de un **bus o mediador central** llamado **Enterprise Service Bus (ESB)**.
- Se apoya en **estándares abiertos** para la **integración y comunicación**, como XML, JSON, SOAP y REST, logrando facilitar la **interoperabilidad** con otros sistemas.
- Principios como el **bajo acoplamiento**, la **alta cohesión** y la **reutilización de servicios** son propios de SOA.
- Cada servicio representa una **funcionalidad de negocio específica** y pueda ser utilizado por múltiples aplicaciones.
- Los servicios son **autónomos, descubribles** (ubicados y utilizados dinámicamente por otros sistemas sin necesidad de conocer previamente su implementación) y pueden **evolucionar** de forma independiente sin afectar al resto del sistema.

SOA (Service Oriented Architecture)

Definición

- Permite que varios servicios simples se **coordinen entre sí para resolver tareas más grandes**. En lugar de que un solo sistema haga todo, **cada servicio cumple una parte del proceso** y se conectan para lograr el resultado final.
- El uso de un **ESB** puede introducir **cierta complejidad y posibles cuellos de botella**, por lo que en arquitecturas modernas a veces se complementa o reemplaza con enfoques como microservicios y comunicación más distribuida.

SOA (Service Oriented Architecture)

Estándares

En SOA, los **estándares y estructuras** se refieren a las formas comunes y reglas definidas que permiten que distintos sistemas puedan comunicarse e integrarse correctamente.

Los estándares **son protocolos, formatos y reglas** ampliamente aceptados que aseguran la **interoperabilidad** entre servicios, sin importar la tecnología utilizada.

Ejemplos:

- **Formatos de datos:** XML, JSON
- **Protocolos de comunicación:** HTTP, HTTPS
- **Estilos o protocolos de servicios:** SOAP, REST
- **Descripción de servicios:** WSDL (describe qué hace un servicio y cómo invocarlo)

Permiten asegurar que todos los servicios “hablen el mismo idioma”.

SOA (Service Oriented Architecture)

Estructuras

En SOA, los **estándares y estructuras** se refieren a las formas comunes y reglas definidas que permiten que distintos sistemas puedan comunicarse e integrarse correctamente.

Las estructuras son **la forma en que se organizan los mensajes y servicios** dentro de la arquitectura:

- **Estructura de mensajes:** cómo se organizan los datos que se envían (por ejemplo, campos, etiquetas, jerarquía en XML/JSON)
 - **Definición de interfaces:** qué operaciones ofrece un servicio, qué parámetros recibe y qué devuelve
 - **Contratos de servicio:** reglas claras sobre cómo interactuar con un servicio (inputs, outputs, errores)
 - **Organización de servicios:** cómo se agrupan o componen para formar procesos más grandes
- Permiten definir **cómo se construyen e intercambian los datos y funcionalidades**.

SOA (Service Oriented Architecture)

No es SOA

- **No es un sistema monolítico:** en SOA la lógica **NO está concentrada** en una única aplicación grande, sino **distribuida en múltiples servicios independientes**. Cada servicio cumple una función específica y puede desarrollarse, desplegarse y mantenerse de forma separada, evitando el fuerte acoplamiento típico de los sistemas monolíticos.
- **No implica comunicación directa rígida:** los servicios **NO se conectan mediante integraciones punto a punto fijas**. En su lugar, utilizan **mecanismos desacoplados** (como un ESB o APIs), lo que permite cambiar, reemplazar o reubicar servicios sin afectar a todos los consumidores.
- **No es dependiente de una sola tecnología:** SOA **NO impone un lenguaje de programación, plataforma o proveedor específico**. Los servicios pueden estar implementados en **distintas tecnologías**, siempre que respeten **estándares de comunicación**, facilitando la integración de sistemas heterogéneos.
- **No es solo uso de servicios web:** el simple hecho de exponer APIs o usar SOAP/REST **NO implica que exista una arquitectura SOA**. SOA requiere principios como **bajo acoplamiento, reutilización, descubrimiento y gobernanza de servicios**.

Bus ESB (Enterprise Service Bus)

Definición

- Su función principal es **gestionar y facilitar la comunicación** entre los **servicios** y los **clientes**, actuando como un mediador inteligente.
- **Recibe las transacciones de los clientes y las dirige hacia el servicio** que las procesará. Una vez que el servicio ha completado su tarea, **el bus recibe la respuesta y la retorna al cliente** correspondiente.
- El bus tiene la responsabilidad de **administrar todos los clientes y servicios** que están conectados a la arquitectura, siendo el componente central y punto focal de la arquitectura SOA.
- Actúa como **punto único de intercambio de mensajes**, evitando que los servicios se conecten directamente entre sí. Esto **reduce el acoplamiento y simplifica la arquitectura**, ya que cada componente solo necesita conocer al bus.
- Actúa como **intermediario entre servicios** que utilizan distintos **protocolos de comunicación** (por ejemplo, HTTP, JMS o FTP). Esto permite que sistemas heterogéneos interactúen sin necesidad de adaptarse directamente unos a otros.

Bus ESB (Enterprise Service Bus)

Definición

- **Detecta fallos en la comunicación o en los servicios y aplica estrategias** como reintentos automáticos, envío a colas de error o redirección a servicios alternativos. Esto mejora la robustez del sistema frente a fallos.
- Es principalmente “**stateless**”: su función es **transportar, transformar y enrutar mensajes**, no almacenar información persistente del negocio. Cada mensaje se procesa de forma independiente.

Bus ESB (Enterprise Service Bus)

Stateless

El estado interno se refiere a **la información propia que un sistema o servicio** mantiene sobre una entidad o proceso en un momento dado.

- En un sistema de pedidos, el estado interno puede ser:
 - Pedido creado
 - Pagado
 - Enviado
- En un usuario:
 - Datos personales
 - Sesión activa/inactiva
 - Historial de acciones

Esa información **vive dentro de los servicios o bases de datos** y representa “cómo está” el sistema en ese instante.

En contraste, el ESB:

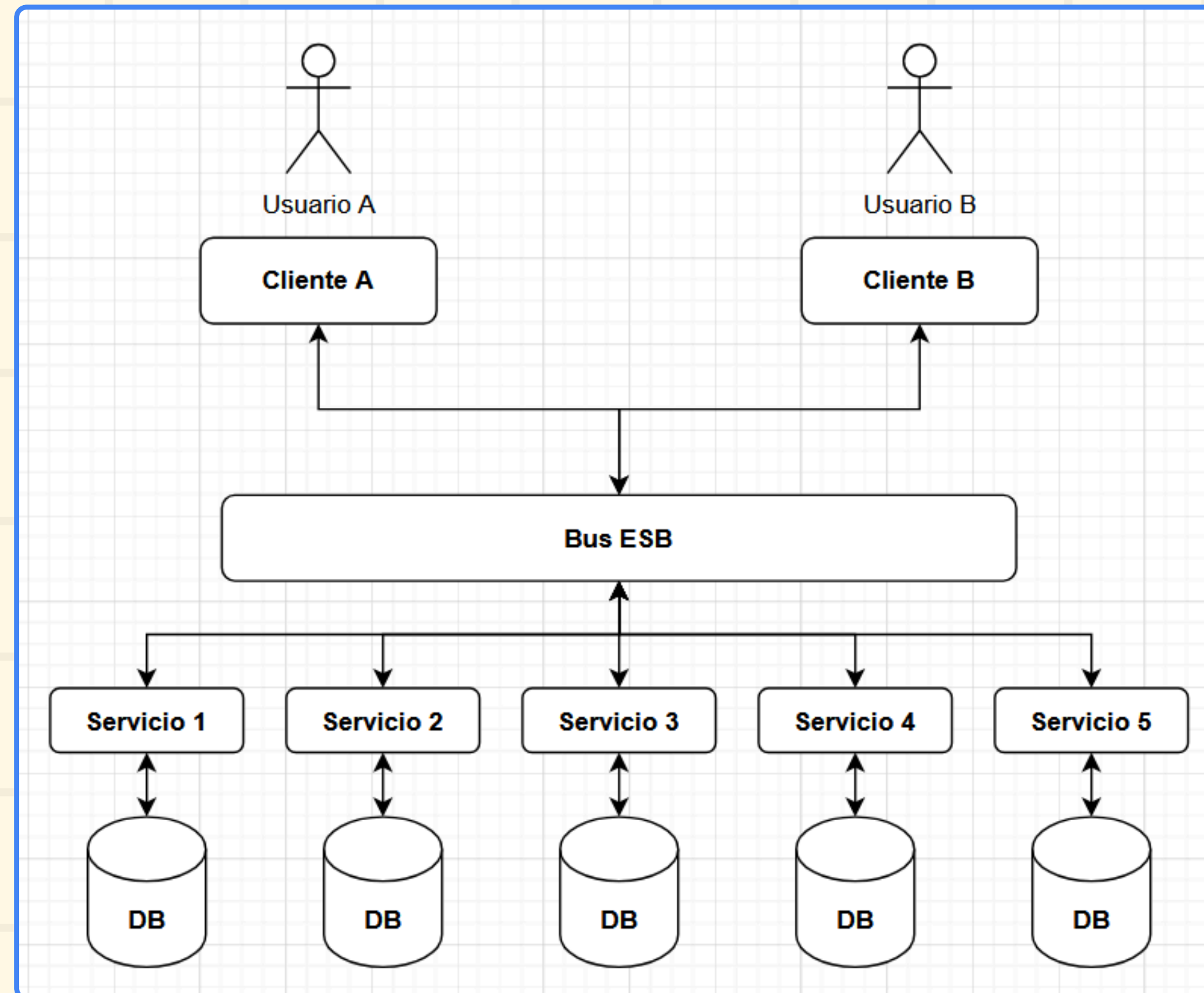
- **No guarda ese estado de negocio** (no sabe si un pedido está pagado o no)
- **Solo maneja mensajes** (por ejemplo: “procesar pago”, “enviar pedido”)
- Puede registrar que ocurrió una acción, pero no mantiene la evolución completa del dato

Bus ESB (Enterprise Service Bus)

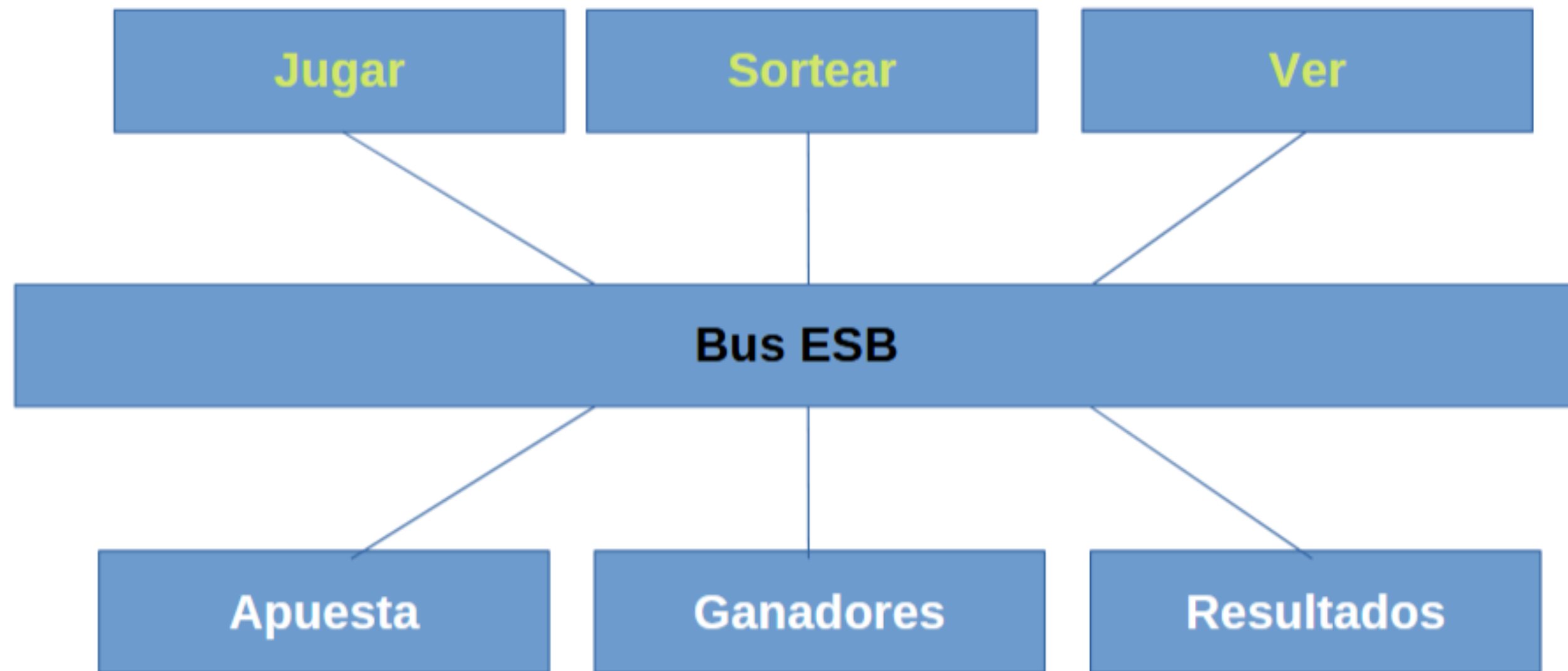
No es ESB

- **No gestiona el estado del negocio:** **NO** almacena ni administra datos persistentes como pedidos, usuarios o transacciones; eso corresponde a los servicios y sus bases de datos.
- **No implementa la lógica de negocio principal:** **NO** ejecuta reglas complejas propias del dominio (por ejemplo, cálculos de negocio o validaciones específicas); **solo coordina y enruta.**
- **No reemplaza a los servicios:** **NO** contiene la funcionalidad completa de la aplicación, sino que **conecta servicios** que sí la implementan.
- **No es una base de datos:** **NO** está diseñado para almacenar información de forma permanente, solo **maneja mensajes** en tránsito o temporalmente.
- **No es un cliente ni una interfaz de usuario:** **NO** interactúa directamente con usuarios finales; actúa en la *capa* de integración.
- **No elimina la necesidad de diseño de arquitectura:** **NO** resuelve automáticamente problemas de acoplamiento o mala organización si los servicios están mal diseñados.
- **No es completamente libre de estado:** aunque puede manejar estado técnico temporal, **NO debe usarse para mantener estado de negocio.**
- **No es equivalente a una API Gateway (aunque puede solaparse):** el ESB tiene un rol más amplio de integración y orquestación, mientras que un API Gateway se enfoca en exponer y gestionar APIs.

SOA (Service Oriented Architecture)



SOA (Service Oriented Architecture)



SOA

Ventajas

- **Reutilización de sistemas existentes:** Permite integrar y reutilizar funcionalidades de sistemas más antiguos sin tener que reescribir el código. Los sistemas se exponen como servicios, lo que permite que las nuevas aplicaciones los utilicen.
- **Bajo nivel de acoplamiento:** Los servicios son independientes entre sí. Si un servicio cambia su lógica interna, los demás servicios que lo utilizan no se ven afectados.
- **Mejora de atributos de calidad:** Provee escalabilidad, mantenibilidad e interoperabilidad al sistema.

Desventajas

- **Bus como punto único de fallo:** Si el bus falla, toda la comunicación entre los servicios se detiene.
- **Complejidad en entornos grandes:** Complejo de implementar y gestionar en sistemas grandes con muchos servicios, ya que el flujo de datos y las interacciones deben ser cuidadosamente monitoreadas y controladas.

Modelo Repositorio

Definición

- Es una arquitectura que se utiliza cuando múltiples aplicaciones necesitan **compartir y acceder a grandes cantidades de datos** de forma **centralizada**.
- Existe un **repositorio de datos central** al que todos los componentes del sistema **pueden acceder directamente**. Se tienen 2 formas de implementación:
- **Modelo Pasivo**: Los componentes que se conectan al repositorio son los que inician las transacciones y solicitan los datos. El repositorio no notifica a los clientes cuando los datos cambian, y los clientes deben actualizarse manualmente.
- **Modelo Activo**: El repositorio notifica a los componentes cuando los datos han sido modificados.
- Toda la información del sistema **se almacena en un único repositorio central**. Esto evita **duplicidades** y asegura que todos los componentes trabajen con los mismos datos actualizados, **reduciendo inconsistencias**.
- El modelo repositorio **no es solo la base de datos**; incluye una *capa* de **acceso/gestión** del repositorio y varios módulos alrededor.

Modelo Repositorio

Definición

- Los componentes dependen directamente de **la estructura del repositorio** (tablas, relaciones, formatos). **Si el esquema cambia, múltiples aplicaciones pueden verse afectadas** y requerir modificaciones.
- Al existir **una única fuente**, es más fácil garantizar reglas de integridad (claves, restricciones, transacciones), lo que mejora **la calidad y coherencia** de la información.
- Si muchos componentes **acceden simultáneamente** al repositorio, este puede convertirse en **un cuello de botella**. Escalar requiere **optimizar o replicar el repositorio**, lo cual puede ser complejo.
- Nuevos componentes pueden **integrarse accediendo al repositorio existente**, sin necesidad de implementar complejos **mecanismos de comunicación** entre sistemas.
- **El repositorio es un punto crítico**; si falla o se vuelve inaccesible, todos los sistemas que dependen de él **dejan de funcionar** o quedan limitados.
- Tareas como **respaldos, recuperación, actualizaciones o migraciones** se realizan sobre **un único repositorio**, lo que simplifica la gestión operativa.

Modelo Repositorio

Acceso a los Datos

El modelo repositorio **no es solo la base de datos**; incluye una *capa* de **acceso/gestión** del repositorio y **varios módulos** alrededor.

- **Capa de acceso al repositorio:** actúa como intermediario entre los módulos y la base de datos. Encapsula cómo se leen/escriben los datos (consultas, transacciones), evitando que cada módulo haga acceso “crudo”.
- **API o interfaz del repositorio:** define operaciones claras (por ejemplo, “obtener usuario”, “guardar pedido”). Los módulos interactúan con esta interfaz en lugar de escribir consultas directamente.
- **Módulos clientes:** son las aplicaciones o componentes que usan el repositorio. No se conectan directamente a la DB, sino a través de la capa del repositorio.
- **Gestión de concurrencia:** controla accesos simultáneos (bloqueos, transacciones) para evitar inconsistencias cuando varios módulos operan al mismo tiempo.
- **Reglas de integridad y validación:** pueden implementarse en esta capa para asegurar que los datos cumplan ciertas condiciones antes de almacenarse.
- **Mecanismos de notificación (modelo activo):** el repositorio puede incluir componentes que avisan a los módulos cuando los datos cambian

Modelo Repositorio

Esquema de Datos

En este contexto, **el esquema se refiere a la estructura o diseño** de cómo están organizados los datos dentro del repositorio.

Incluye, por ejemplo:

- **Tablas o colecciones** (dónde se guardan los datos)
- **Campos/atributos** (qué información contiene cada registro)
- **Tipos de datos** (texto, número, fecha, etc.)
- **Relaciones entre datos** (por ejemplo, un pedido asociado a un cliente)
- **Restricciones** (claves primarias, foráneas, reglas de integridad)

Cuando se dice que hay **alto acoplamiento al esquema**, significa que **las aplicaciones dependen de esa estructura**.

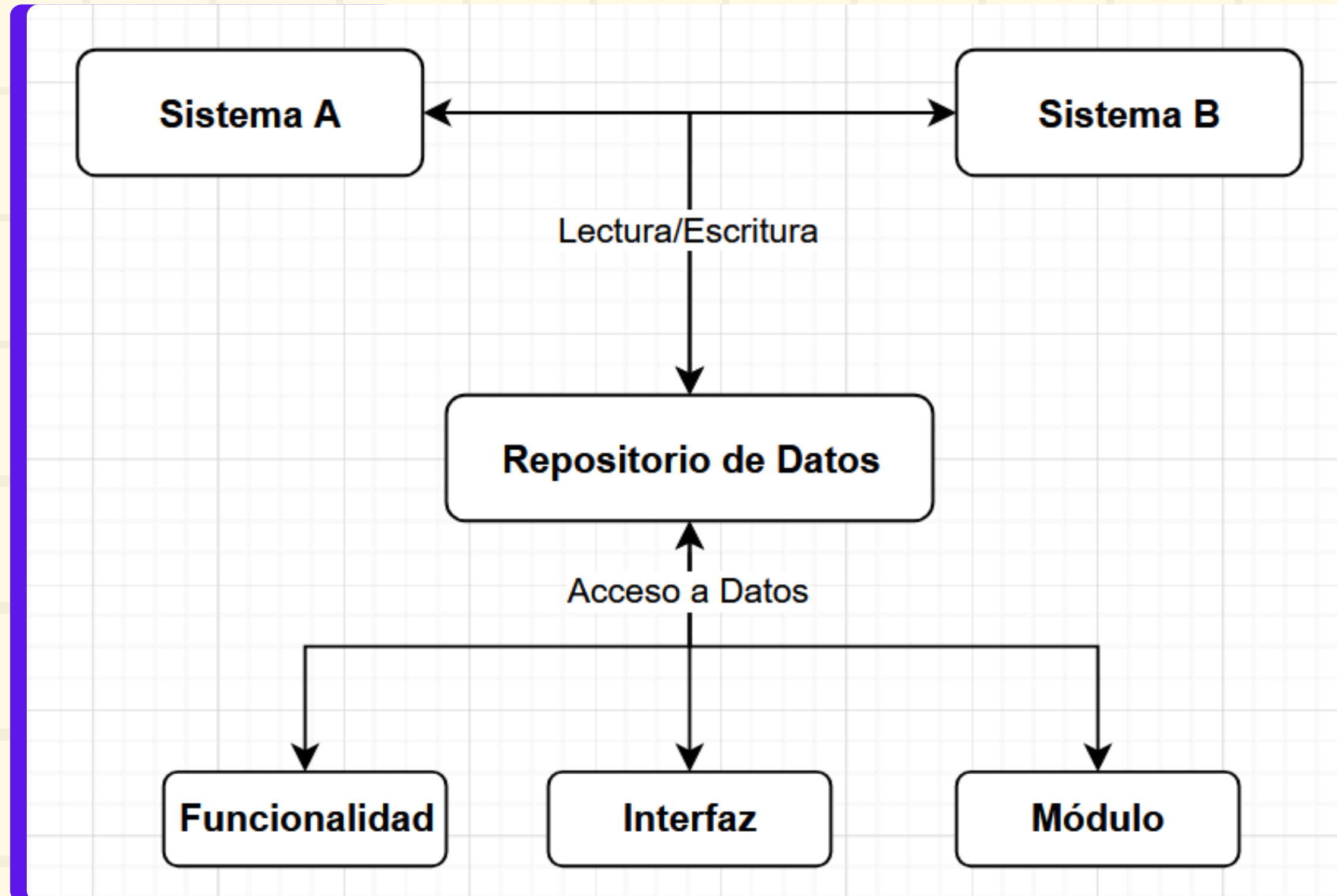
Si cambias el esquema (por ejemplo, renombrar un campo o dividir una tabla), los sistemas que usan esos datos pueden fallar o necesitar cambios.

Modelo Repositorio

No es Repositorio

- **No es solo una base de datos:** **NO** se limita al almacenamiento físico de datos; incluye una capa lógica que gestiona el acceso y uso del repositorio.
- **No es una arquitectura distribuida por servicios:** **NO** divide la lógica en servicios independientes que se comunican entre sí (como en SOA o microservicios).
- **No elimina el acoplamiento:** los módulos siguen **dependiendo del esquema de datos**, por lo que cambios en el repositorio afectan a varios componentes.
- **No abstrae completamente el modelo de datos:** aunque existe una *capa* de acceso, los módulos **suelen conocer la estructura del repositorio**.
- **No gestiona lógica de negocio compleja:** su foco está en el **almacenamiento y acceso**, **NO** en procesar **reglas de negocio** avanzadas en el repositorio (clientes, módulos).
- **No es altamente escalable por defecto:** al centralizar los datos, puede convertirse en un **cuello de botella** si crece la demanda.
- **No garantiza independencia total entre componentes:** los módulos **comparten el mismo repositorio**, lo que genera dependencia indirecta entre ellos.
- **No es equivalente a un ESB o middleware de integración:** **NO** coordina servicios ni enruta mensajes, su rol principal es la gestión centralizada de datos.

Modelo Repositorio



Modelo Repositorio

Ventajas

- **Eficiencia para compartir datos:** Al tener un lugar centralizado para el almacenamiento, es un modo muy eficiente para que las aplicaciones compartan información entre sí.
- **Administración centralizada:** La gestión de los datos, la seguridad y las políticas de acceso se administran en un solo lugar.
- **Seguridad:** Se pueden implementar políticas de acceso estrictas y uniformes para todas las aplicaciones y usuarios que interactúan con los datos.
- **Escalabilidad:** El almacenamiento puede ser centralizado o distribuido, permitiendo crecer para manejar grandes volúmenes de datos. Aunque la base de datos es un punto central, puede ser escalada para soportar más carga y almacenamiento.
- **Mantenibilidad:** Al tener una única fuente, se asegura que todas las aplicaciones que acceden a los datos trabajen con la misma información, evitando inconsistencias.

Desventajas

- **Punto único de falla:** Como el repositorio es un componente central, si este falla, todo el sistema deja de funcionar.
- **Dificultad de cambio del modelo de datos:** Cualquier modificación en la estructura de los datos afectará a todas las aplicaciones que lo utilizan.
- **Complejidad en la administración:** Puede volverse muy compleja en sistemas grandes, lo que puede dificultar la evolución del sistema.

Paso 1: Descargar imagen

Descripción

- Desde la máquina ejecutar: `docker run -d -p 5000:5000 jrgiadach/soabus:v1`
- Esto **descarga la imagen de Docker**, crea y ejecuta un contenedor a partir de esta en segundo plano, mapeando el **puerto 5000** de tu máquina a el **puerto 5000** del contenedor.

```
bruno@bruno-IdeaPad-1-14I1L7:~/Documentos/SOA$ docker run -d -p 5000:5000 jrgiadach/soabus:v1
Unable to find image 'jrgiadach/soabus:v1' locally
v1: Pulling from jrgiadach/soabus
31e352740f53: Pull complete
68c8dd1577de: Pull complete
2af2ea38db33: Pull complete
c8c07c0fde10: Pull complete
a86f53a76731: Pull complete
Digest: sha256:57b3cb5604c647084756d0582fe7a748fc314d2958599bd5679e985d022780d1
Status: Downloaded newer image for jrgiadach/soabus:v1
3f73ad7e039958a5928735be62769ada409971111d83b28378a8a05b65ac6d92
```


Paso 2: Componentes

Componente Cliente

Es quien inicia la comunicación.

- **Se conecta al bus** (connect_to_bus)
- **Envía solicitudes a un servicio** (send_message)
- **Espera y recibe respuestas** (receive_message)

Se encarga de solicitar la ejecución de una funcionalidad (por ejemplo, pedir al servicio que espere X segundos).

Qué hace en el código de ejemplo:

- Toma un input del usuario
- Lo envía al servicio "servi"
- Espera la respuesta del bus
- Muestra el resultado

Es el consumidor de servicios.

 soa_client.py

 soa_lib.py

 soa_service.py

```
1 from soa_lib import connect_to_bus, send_message, receive_message
2
3 sock = connect_to_bus()
4
5 try:
6     while True:
7         entrada = input('\nIngrese segundos de espera (o "q" para salir): ')
8
9         if entrada.lower() == 'q':
10             break
11
12         if not entrada.isdigit():
13             print("¡Error! Por favor ingrese solo números.")
14             continue
15
16         # Enviar el número como string al servicio "servi"
17         send_message(sock, "servi", entrada)
18
19         print(f"Esperando respuesta del servicio ({entrada}s)...")
20         data = receive_message(sock)
21
22         if data:
23             # Mostrar el mensaje (quitando los 5 caracteres del nombre del servicio)
24             print(f"Respuesta recibida: {data[5:].decode()}")
25
26 finally:
27     print('Cerrando conexión')
28     sock.close()
```


Paso 2: Componentes

Componente Servicio

Es quien procesa la lógica.

- Se registra en el bus con "sinit"
- Espera solicitudes del cliente
- Ejecuta la operación
- Devuelve una respuesta

Se encarga de implementar la funcionalidad (la lógica de negocio).

Qué hace en el código de ejemplo:

- Se registra como "servi" en el ESB
- Recibe un número
- Simula trabajo (sleep)
- Responde "OK" o error

Es el proveedor del servicio.

 soa_client.py

 soa_lib.py

 soa_service.py

```
1 from soa_lib import connect_to_bus, send_message, receive_message
2 import time
3
4 sock = connect_to_bus()
5
6 try:
7     # 1. Registro inicial (sinit)
8     print("Registrando servicio 'servi'...")
9     send_message(sock, "sinit", "servi")
10
11     # 2. Procesar respuesta del sinit
12     init_data = receive_message(sock)
13     print(f"Confirmación de bus recibida: {init_data!r}")
14     print("Servicio listo para recibir transacciones.\n")
15
16     # 3. Bucle principal de trabajo
17     while True:
18         data = receive_message(sock)
19         if not data:
20             print("Conexión cerrada por el bus.")
21             break
22
23         # Extraer el payload (salta los 5 caracteres del nombre del servicio)
24         mensaje = data[5:].decode()
25         print(f"Mensaje recibido del cliente: '{mensaje}'")
26         try:
27             segundos = int(mensaje)
28             print(f"Simulando trabajo: durmiendo {segundos}s...")
29             time.sleep(segundos)
```

Paso 2: Componentes

Componente Extra: Librería Compartida

Es una librería compartida que abstrae la comunicación.

- Maneja sockets
- Define el protocolo de mensajes
- Evita duplicar código en cliente y servicio

Se encarga de encapsular la forma de comunicarse con el bus.

Qué hace:

- connect_to_bus → abre conexión
- send_message → arma el mensaje (nnnnn + contenido)
- receive_message → lee y reconstruye el mensaje

Es la capa técnica de comunicación.

 soa_client.py

 soa_lib.py

 soa_service.py

```

1  import socket
2
3  def connect_to_bus(host='localhost', port=5000):
4      sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
5      print(f"Conectando a {host} en puerto {port}...")
6      sock.connect((host, port))
7      return sock
8
9  def send_message(sock, service_name, payload):
10     # concatenar el nombre del servicio y el payload
11     content = service_name.encode() + payload.encode()
12     length = str(len(content)).zfill(5)
13     message = length.encode() + content
14     sock.sendall(message)
15
16 def receive_message(sock):
17     raw_len = sock.recv(5)
18     if not raw_len:
19         return None
20     try:
21         amount_expected = int(raw_len)
22     except ValueError:
23         return None
24
25     data = b''
26     while len(data) < amount_expected:
27         chunk = sock.recv(amount_expected - len(data))
28         if not chunk: break
29         data += chunk
30     return data
31

```

Paso 2: Componentes

Relación

Cliente → (soa_lib) → ESB → (soa_lib) → Servicio

Servicio → (soa_lib) → ESB → (soa_lib) → Cliente

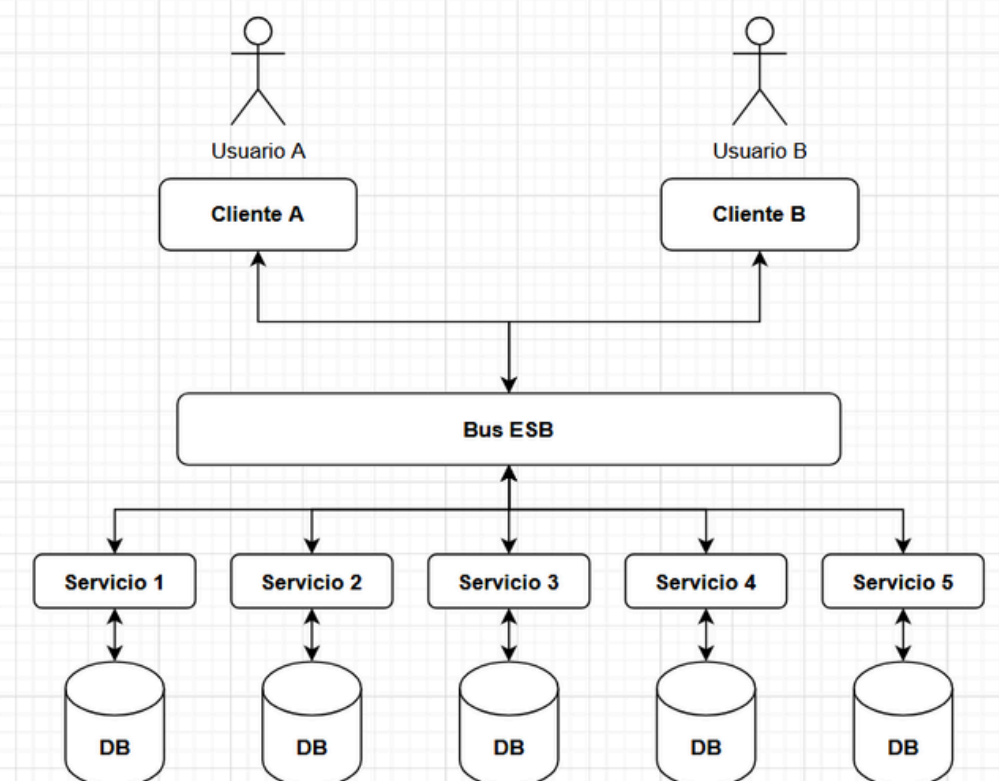
- **Cliente:** pide
- **Servicio:** ejecuta
- **soa_lib:** conecta y estandariza la comunicación

Sin soa_lib, cliente y servicio tendrían que implementar manualmente sockets y protocolo; sin servicio no hay lógica; sin cliente no hay quien consuma.

 soa_client.py

 soa_lib.py

 soa_service.py



Paso 3: Iniciar Servicios

Descripción

- Desde la máquina ejecutar el servicio. Esto provoca que el servicio se **registre en el bus**, indicando que esta operativo y listo para recibir solicitudes.
- El bus siempre esta “escuchando” o **recibiendo solicitudes**, en este caso **sinitOKservi**, indicando que se inicio el servicio correctamente.

```
(.venv) bruno@bruno-IdeaPad-1-14IJL7:~/Documentos/SOA$ python soa_service.py
Conectando a localhost en puerto 5000...
Registrando servicio 'servi'...
Confirmación de bus recibida: b'sinitOKservi'
Servicio listo para recibir transacciones.
```

Paso 3: Iniciar Clientes

Descripción

- Desde la máquina ejecutar el cliente. Esto provoca que el servicio se **registre en el bus**, indicando que esta operativo y listo para enviar solicitudes.
- El bus siempre esta “escuchando” o **recibiendo solicitudes**.

```
(.venv) bruno@bruno-IdeaPad-1-14IJL7:~/Documentos/SOA$ python soa_client.py
Conectando a localhost en puerto 5000...
Ingrese segundos de espera (o "q" para salir):
```


Paso 3: Bus ESB

Descripción

- El bus se encuentra ya levantado (ver Paso 1). Para acceder a sus logs se puede usar: **docker logs -f <container_id o nombre>**
- El bus siempre esta “escuchando” o **recibiendo solicitudes**. En este caso, se puede llevar registro de los servicios y clientes que se han conectado, enviado solicitudes, etc. En este caso se recibe y envía la solicitud del servicio registrado

```
bruno@bruno-IdeaPad-1-14IJL7:~/Documentos/SOA$ docker logs -f 3f73ad7e0399
Wed Apr 29 00:10:00 -04 2026
29/04 00:10:00: /home/soa/bus: waiting for data on port TCP 5000 (debug on)
29/04 01:26:29: ... Evento de connexion
29/04 01:26:29: One more <1> Socket: 5
29/04 01:26:29: ... Evento en Socket 5
29/04 01:26:29: R5 <00010sinitservi> [0]
29/04 01:26:29: S5 <00012sinitOKservi> [0]
29/04 01:31:41: ... Evento de connexion
29/04 01:31:41: One more <2> Socket: 6
```

Paso 4: Enviar solicitud

Estructura de la transacción (request)

Se tiene definido un **protocolo de comunicación** para interactuar con el servicio (ESB) en el **puerto 5000**. La idea es que cada mensaje tenga un **formato fijo** para que el sistema pueda interpretarlo correctamente:

- **nnnnnSSSSSDATOS**
- **nnnnn**: indica la cantidad de caracteres que vienen después (todo lo demás del mensaje).
- **SSSSS**: nombre del servicio (exactamente 5 caracteres).
- **DATOS**: información que necesita el servicio para operar.

```
Ingrese segundos de espera (o "q" para salir): 5
Esperando respuesta del servicio (5s)...
Respuesta recibida: OKOK
```


Paso 4: Enviar solicitud

Estructura de la transacción (request)

Cómo se interpreta el ejemplo:

- **00006servi5**
- **00006** → vienen 6 caracteres después
- **servi** → servicio a ejecutar
- **5** → datos (segundos)

```
29/04 01:42:24: ... Evento en Socket 6
29/04 01:42:24: R6 <00006servi5> [1]
29/04 01:42:24: S5 <00006servi5> [1]
29/04 01:42:29: ... Evento en Socket 5
29/04 01:42:29: R5 <00007servi0K> [0]
```

Paso 4: Enviar solicitud

Estructura de la respuesta

Se tiene definido un **protocolo de comunicación** para interactuar con el servicio (ESB) en el **puerto 5000**. La idea es que cada mensaje tenga un **formato fijo** para que el sistema pueda interpretarlo correctamente:

- **nnnnnSSSSSRESULTADODATOS**
- **nnnnn**: largo total del contenido restante
- **SSSSS**: nombre del servicio
- **RESULTADO**:
 - **OK** → operación exitosa
 - **NK** → error
- **DATOS**: resultado o mensaje devuelto

```
Mensaje recibido del cliente: '5'
Simulando trabajo: durmiendo 5s...
Respuesta 'OK' enviada.
```

Paso 4: Enviar solicitud

Estructura de la respuesta

Se tiene definido un **protocolo de comunicación** para interactuar con el servicio (ESB) en el **puerto 5000**. La idea es que cada mensaje tenga un **formato fijo** para que el sistema pueda interpretarlo correctamente:

- **00007serviOK**
- **00007** → 7 caracteres después
- **servi** → servicio ejecutado
- **OK** → operación correcta

```
29/04 01:42:24: ... Evento en Socket 6
29/04 01:42:24: R6 <00006servi5> [1]
29/04 01:42:24: S5 <00006servi5> [1]
29/04 01:42:29: ... Evento en Socket 5
29/04 01:42:29: R5 <00007serviOK> [0]
```

Paso 4: Enviar solicitud

Librería Extra Compartida

Es una **librería de apoyo** que encapsula toda la lógica de **comunicación con el bus (ESB)**. Su objetivo es que cliente y servicio no tengan que preocuparse por los detalles técnicos de **sockets** ni del **formato** de los mensajes.

Qué hace cada función:

- **connect_to_bus()** Crea un socket TCP
 - Se conecta a localhost:5000
 - Devuelve la conexión lista para usar

```
def connect_to_bus(host='localhost', port=5000):
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    print(f"Conectando a {host} en puerto {port}...")
    sock.connect((host, port))
    return sock

def send_message(sock, service_name, payload):
    # concatenar el nombre del servicio y el payload
    content = service_name.encode() + payload.encode()
    length = str(len(content)).zfill(5)
    message = length.encode() + content
    sock.sendall(message)

def receive_message(sock):
    raw_len = sock.recv(5)
    if not raw_len:
        return None
    try:
        amount_expected = int(raw_len)
    except ValueError:
        return None

    data = b''
    while len(data) < amount_expected:
        chunk = sock.recv(amount_expected - len(data))
        if not chunk: break
        data += chunk
    return data
```

Paso 4: Enviar solicitud

Librería Extra Compartida

Es una **librería de apoyo** que encapsula toda la lógica de **comunicación con el bus (ESB)**. Su objetivo es que cliente y servicio no tengan que preocuparse por los detalles técnicos de **sockets** ni del **formato** de los mensajes.

Qué hace cada función:

- **send_message(sock, service_name, payload)**
 - Construye el mensaje según el protocolo: nnnnn + servicio + datos
 - Calcula el largo (nnnnn)
 - Envía el mensaje completo al bus

```
def connect_to_bus(host='localhost', port=5000):  
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  
    print(f"Conectando a {host} en puerto {port}...")  
    sock.connect((host, port))  
    return sock  
  
def send_message(sock, service_name, payload):  
    # concatenar el nombre del servicio y el payload  
    content = service_name.encode() + payload.encode()  
    length = str(len(content)).zfill(5)  
    message = length.encode() + content  
    sock.sendall(message)  
  
def receive_message(sock):  
    raw_len = sock.recv(5)  
    if not raw_len:  
        return None  
    try:  
        amount_expected = int(raw_len)  
    except ValueError:  
        return None  
  
    data = b''  
    while len(data) < amount_expected:  
        chunk = sock.recv(amount_expected - len(data))  
        if not chunk: break  
        data += chunk  
    return data
```


Paso 4: Enviar solicitud

Librería Extra Compartida

Es una **librería de apoyo** que encapsula toda la lógica de **comunicación con el bus (ESB)**. Su objetivo es que cliente y servicio no tengan que preocuparse por los detalles técnicos de **sockets** ni del **formato** de los mensajes.

Qué hace cada función:

- **receive_message(sock)**
 - Lee primero los 5 bytes del largo
 - Luego recibe exactamente esa cantidad de datos
 - Devuelve el mensaje completo recibido

```
def connect_to_bus(host='localhost', port=5000):
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    print(f"Conectando a {host} en puerto {port}...")
    sock.connect((host, port))
    return sock

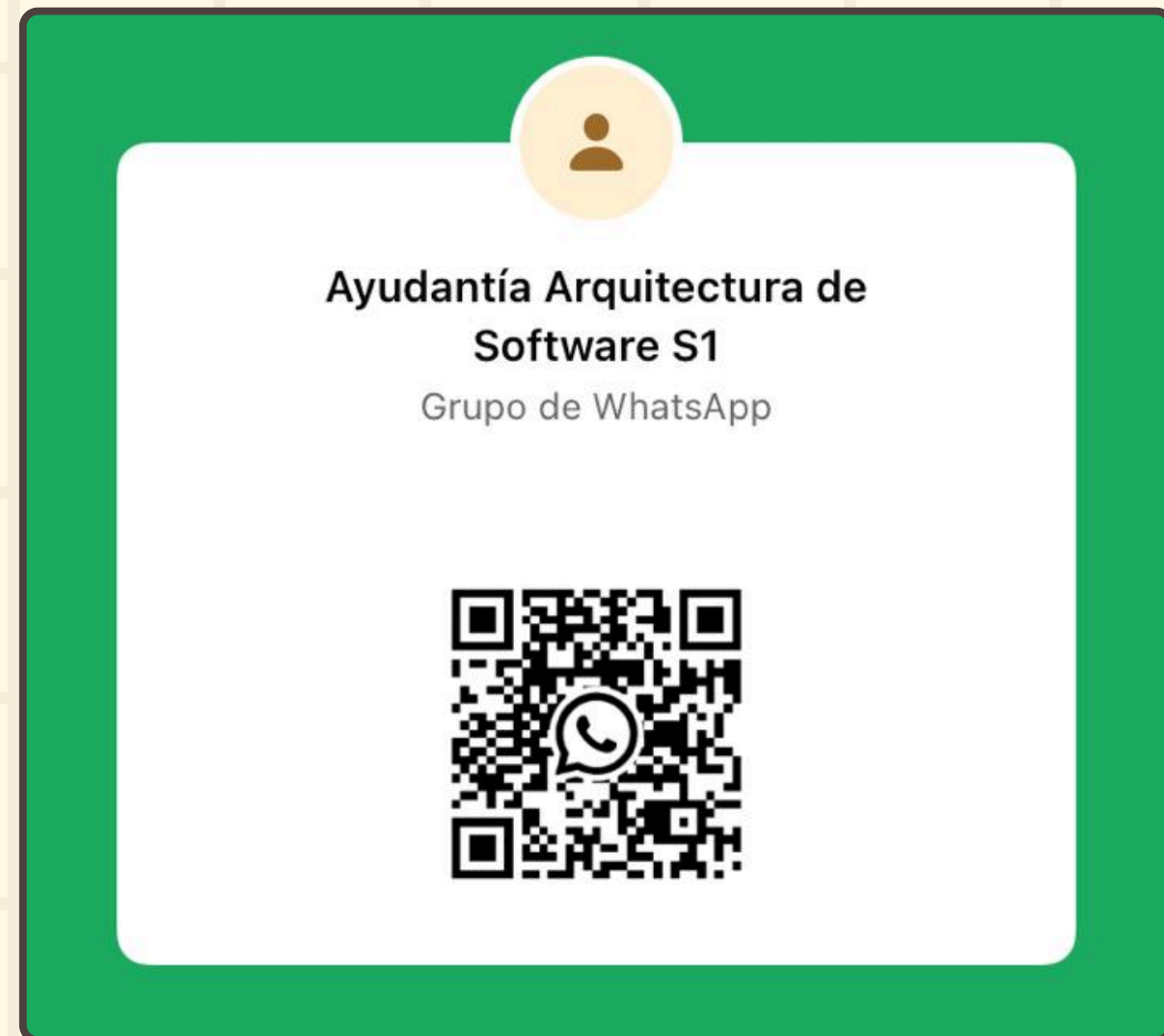
def send_message(sock, service_name, payload):
    # concatenar el nombre del servicio y el payload
    content = service_name.encode() + payload.encode()
    length = str(len(content)).zfill(5)
    message = length.encode() + content
    sock.sendall(message)

def receive_message(sock):
    raw_len = sock.recv(5)
    if not raw_len:
        return None
    try:
        amount_expected = int(raw_len)
    except ValueError:
        return None

    data = b''
    while len(data) < amount_expected:
        chunk = sock.recv(amount_expected - len(data))
        if not chunk: break
        data += chunk
    return data
```


MEDIOS DE COMUNICACIÓN

Medios de Comunicación



+56 9 4579 0672



bruno.rosales@mail.udp.cl

GRUPO DE WHATSAPP



DRIVE AYUDANTÍAS

